

[Learn the Basics](#) || [Quickstart](#) || [Tensors](#) || [Datasets & DataLoaders](#) || [Transforms](#) || **Build Model** || [Autograd](#) || [Optimization](#) || [Save & Load Model](#)

Build the Neural Network

Created On: Feb 09, 2021 | Last Updated: Jan 24, 2025 | Last Verified: Not Verified

Neural networks comprise of layers/modules that perform operations on data. The [torch.nn](#) namespace provides all the building blocks you need to build your own neural network. Every module in PyTorch subclasses the [nn.Module](#). A neural network is a module itself that consists of other modules (layers). This nested structure allows for building and managing complex architectures easily.

In the following sections, we'll build a neural network to classify images in the FashionMNIST dataset.

```
import os
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
```

Get Device for Training

We want to be able to train our model on an [accelerator](#) such as CUDA, MPS, MTIA, or XPU. If the current accelerator is available, we will use it. Otherwise, we use the CPU.

```
device = torch.accelerator.current_accelerator().type if torch.accelerator.is_
print(f"Using {device} device")
```



Out:

Using cuda device

Define the Class

We define our neural network by subclassing `nn.Module`, and initialize the neural network layers in `__init__`. Every `nn.Module` subclass implements the operations on input data in the `forward` method.

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
```

We create an instance of `NeuralNetwork`, and move it to the `device`, and print its structure.

```
model = NeuralNetwork().to(device)
print(model)
```

Out:

```
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
```

To use the model, we pass it the input data. This executes the model's `forward`, along with some background operations. Do not call `model.forward()` directly!

Calling the model on the input returns a 2-dimensional tensor with `dim=0` corresponding to each output of 10 raw predicted values for each class, and `dim=1` corresponding to the individual values of each output. We get the prediction probabilities by passing it through an instance of the `nn.Softmax` module.

```
X = torch.rand(1, 28, 28, device=device)
logits = model(X)
pred_probab = nn.Softmax(dim=1)(logits)
y_pred = pred_probab.argmax(1)
print(f"Predicted class: {y_pred}")
```

Out:

```
Predicted class: tensor([6], device='cuda:0')
```

Model Layers

Let's break down the layers in the FashionMNIST model. To illustrate it, we will take a sample minibatch of 3 images of size 28x28 and see what happens to it as we pass it through the network.

```
input_image = torch.rand(3,28,28)
print(input_image.size())
```

Out:

```
torch.Size([3, 28, 28])
```

nn.Flatten

We initialize the [nn.Flatten](#) layer to convert each 2D 28x28 image into a contiguous array of 784 pixel values (the minibatch dimension (at dim=0) is maintained).

```
flatten = nn.Flatten()
flat_image = flatten(input_image)
print(flat_image.size())
```

Out:

```
torch.Size([3, 784])
```

nn.Linear

The [linear layer](#) is a module that applies a linear transformation on the input using its stored weights and biases.

```
layer1 = nn.Linear(in_features=28*28, out_features=20)
hidden1 = layer1(flat_image)
print(hidden1.size())
```

Out:

```
torch.Size([3, 20])
```

nn.ReLU

Non-linear activations are what create the complex mappings between the model's inputs and outputs. They are applied after linear transformations to introduce *nonlinearity*, helping neural networks learn a wide variety of phenomena.

In this model, we use [nn.ReLU](#) between our linear layers, but there's other activations to introduce non-linearity in your model.

```
print(f"Before ReLU: {hidden1}\n\n")
hidden1 = nn.ReLU()(hidden1)
print(f"After ReLU: {hidden1}")
```

Out:

```
Before ReLU: tensor([[ 0.1654, -0.1826, -0.2569,  0.2734, -0.4906, -0.0605, -0.2224, -0.1978, -0.3247,  0.1481, -0.0760,  0.6130, -0.9421,  0.3206, -0.4453,  0.1539,  0.4268,  0.4747],
                    [ 0.1926, -0.4804, -0.4498,  0.3072, -0.6464,  0.0892, -0.2870, -0.4804,  0.3291, -0.1148, -0.5243,  0.0230,  0.0813,  0.5538, -0.6838,  0.1853, -0.4386,  0.6457,  0.2448,  0.1899],
                    [ 0.1613, -0.0539, -0.6374, -0.1727, -0.6171,  0.1910, -0.1489, -0.1148,  0.2227, -0.3006, -0.2888, -0.1439,  0.1498,  0.2785, -0.4948,  0.0944, -0.4710,  0.8184,  0.1558,  0.7225]], grad_fn=<AddmmBackward0>)
```

```
After ReLU: tensor([[0.1654, 0.0000, 0.0000, 0.2734, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.1481, 0.0000, 0.6130, 0.0000, 0.3206, 0.0000, 0.1539, 0.4268, 0.4747],
                   [0.1926, 0.0000, 0.0000, 0.3072, 0.0000, 0.0892, 0.0000, 0.0000, 0.3206, 0.0000, 0.0000, 0.0230, 0.0813, 0.5538, 0.0000, 0.1853, 0.0000, 0.6457, 0.2448, 0.1899],
                   [0.1613, 0.0000, 0.0000, 0.0000, 0.0000, 0.1910, 0.0000, 0.0000, 0.2227, 0.0000, 0.0000, 0.0000, 0.1498, 0.2785, 0.0000, 0.0944, 0.0000, 0.8184, 0.1558, 0.7225]], grad_fn=<ReluBackward0>)
```

nn.Sequential

[nn.Sequential](#) is an ordered container of modules. The data is passed through all the modules in the same order as defined. You can use sequential containers to put together a quick network like `seq_modules`.

```
seq_modules = nn.Sequential(  
    flatten,  
    layer1,  
    nn.ReLU(),  
    nn.Linear(20, 10)  
)  
input_image = torch.rand(3,28,28)  
logits = seq_modules(input_image)
```

nn.Softmax

The last linear layer of the neural network returns *logits* - raw values in $[-\infty, \infty]$ - which are passed to the [nn.Softmax](#) module. The logits are scaled to values $[0, 1]$ representing the model's predicted probabilities for each class. `dim` parameter indicates the dimension along which the values must sum to 1.

```
softmax = nn.Softmax(dim=1)  
pred_probab = softmax(logits)
```

Model Parameters

Many layers inside a neural network are *parameterized*, i.e. have associated weights and biases that are optimized during training. Subclassing `nn.Module` automatically tracks all fields defined inside your model object, and makes all parameters accessible using your model's `parameters()` or `named_parameters()` methods.

In this example, we iterate over each parameter, and print its size and a preview of its values.

```
print(f"Model structure: {model}\n\n")

for name, param in model.named_parameters():
    print(f"Layer: {name} | Size: {param.size()} | Values : {param[:2]} \n")
```

Out:

```
Model structure: NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
```

```
Layer: linear_relu_stack.0.weight | Size: torch.Size([512, 784]) | Values : tensor(
  [ 0.0288,  0.0211,  0.0303, ...,  0.0218,  0.0140, -0.0321]],
  device='cuda:0', grad_fn=<SliceBackward0>)
```

[Send Feedback](#)

```
Layer: linear_relu_stack.0.bias | Size: torch.Size([512]) | Values : tensor([
```

```
Layer: linear_relu_stack.2.weight | Size: torch.Size([512, 512]) | Values : tensor(
  [ 0.0263,  0.0274, -0.0320, ..., -0.0310,  0.0425, -0.0407]],
  device='cuda:0', grad_fn=<SliceBackward0>)
```

```
Layer: linear_relu_stack.2.bias | Size: torch.Size([512]) | Values : tensor([
```

Built with the [PyData Sphinx Theme](#) 0.15.4.

Docs

Access comprehensive
developer documentation
for PyTorch

[View Docs](#)

Tutorials

Get in-depth tutorials for
beginners and advanced
developers

[View Tutorials](#)

Resources

Find development
resources and get your
questions answered

[View Resources](#)

Stay in touch for updates, event info, and the latest news

First Name*

Last Name*

Email*

Select Country*

SUBMIT

By submitting this form, I consent to receive marketing emails from the LF and its projects regarding their events, training, research, developments, and related announcements. I understand that I can unsubscribe at any time using the links in the footers of the emails I receive. [Privacy Policy](#).

in

© PyTorch. Copyright © The Linux Foundation®. All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For more information, including terms of use, privacy policy, and trademark usage, please see our [Policies](#) page. [Trademark Usage](#). [Privacy Policy](#).